Ahsanullah University of Science and Technology

Department of Electrical & Electronic Engineering

Assignment on Experiment No. 08

Course No	: **EEE 4154**
Course Name	: **Power System II Lab**
Date of Submission	: 30/09/2026

Experiment Name : **Load Flow Study Using MATLAB (Gauss-Seidel Method)**

Submitted By:

Id	: 20220205048
	: 20220205050
	: 20220205057
	: 20220205059
Section	: A (A2)
Year	: 4th
Semester	: 1st
Group	: 3

Objective

To develop a MATLAB program that solves the load flow problem of a power system by the Gauss-Seidel iterative method, following the procedure of the lab manual. The program takes the bus and line data from the user, forms the bus admittance matrix (Ybus), finds every unknown bus quantity, the slack bus power, the line-to-line power flows and the number of iterations, writes the results to a text file, and plots the convergence of the solution and the bus voltage angles through the iterations.

Theory

Load flow (power flow) analysis is the steady-state study of a power system. It gives the magnitude and phase angle of the voltage at each bus and the real and reactive power flowing in each line. It is needed for the best operation of an existing system and for planning its future expansion.

- **Bus types and net injected power**

At each bus four quantities are involved: P, Q, |V| and δ. Two are specified and two are found by the load flow. P and Q are the net power injected into the network by the bus:

$$P_i = P_Gi - P_Di\,, \quad Q_i = Q_Gi - Q_Di$$

where the subscript G means generation and D means demand (load) at that bus. A positive value means the bus supplies power to the network and a negative value means it draws power from the network.

Bus type	Known	Unknown
Slack / Swing (bus 1)	|V|, δ = 0	P, Q
PV / Generator	P, |V|	Q, δ
PQ / Load	P, Q	|V|, δ

The slack bus supplies the difference between total load plus losses and the specified generation, because the losses are not known until the load flow is solved. Its angle is the reference (0°).

- **Gauss-Seidel equations**

The complex power injected at bus i is $S_i^* = P_i - jQ_i = V_i^* \Sigma Y_{ik}V_k$. Solving for V_i gives the update used for every bus except the slack bus:

$$V_i(\text{new}) = (1/Y_{ii})\,[\,(P_i - jQ_i)/V_i^* - \Sigma(k\neq i)\,Y_{ik}V_k\,] \qquad \ldots (2)$$

For a PV bus Q is unknown, so it is calculated first from the latest voltages; then only the angle of the new voltage is kept and |V| stays at its specified value:

$$Q_i(\text{new}) = -\text{Imag}\,\{\,V_i^* \,\Sigma\, Y_{ik}V_k\,\} \qquad \ldots (1)$$

Each new value is used immediately for the next bus (this is what distinguishes Gauss-Seidel from the Gauss/Jacobi method). After convergence the slack bus power and line flows are found from:

$$S_1 = V_1\,(\,\Sigma\, Y_{1k}V_k\,)^* \;\rightarrow\; P_1 = \text{Real}\{S_1\},\; Q_1 = \text{Imag}\{S_1\} \qquad \ldots (3), (4)$$

$$S_{ik} = V_i\,[\,(V_i - V_k)\,y_{ik}\,]^* \qquad \ldots (5)$$

MATLAB Code

```
clear; clc; close all;

no_bus=input('Total no of bus is: ')

P = zeros(1,no_bus); Q = zeros(1,no_bus);
V = zeros(1,no_bus); delta = zeros(1,no_bus);

for i = 1:no_bus
 fprintf(sprintf('--- Bus %d ---\n', i));
 P(i) = input(sprintf('P(%d) in pu (type inf if unknown): ', i));
 Q(i) = input(sprintf('Q(%d) in pu (type inf if unknown): ', i));
 V(i) = input(sprintf('V(%d) in pu (type inf if unknown): ', i));
 delta(i) = input(sprintf('delta(%d) in rad (type inf if unknown): ', i));
 end


V(isinf(V)) = 1;
delta(isinf(delta)) = 0;
 Vc = V .* exp(1j*delta);

 % Safety check:
for i = 2:no_bus
 if isinf(P(i))
 error(['Bus %d: P was entered as inf, but only the slack bus ' ...
'(bus 1) may have an unknown P. Every PV/PQ bus needs a ' ...
 'real P value -- re-run and enter the actual number.'], i);
 end
 end

isPV = isinf(Q);

 %% YBus formation
Y = zeros(no_bus);
for i = 1:no_bus
 for k = i+1:no_bus
fprintf(sprintf('--- Line %d-%d (type inf,inf if no line) ---\n', i, k));
 r = input(sprintf('R(%d,%d) in pu: ', i, k));
 x = input(sprintf('X(%d,%d) in pu: ', i, k));
 y = 1/(r + 1j*x);
 Y(i,k) = Y(i,k) - y; Y(k,i) = Y(i,k);
 Y(i,i) = Y(i,i) + y; Y(k,k) = Y(k,k) + y;
 end
 end

 % Safety check:1 line connect
for i = 1:no_bus
 if Y(i,i) == 0
 error(['Bus %d has no connecting line (Y(%d,%d)=0), so its ' ...
 'voltage equation divides by zero. Check the R/X you ' ...
 'entered for every line touching this bus.'], i, i, i);
 end
end

fprintf('\n==================== BUS ADMITTANCE MATRIX (YBus) in pu ====================\n');
fprintf('%8s', '');
for k = 1:no_bus
 fprintf('%22s', sprintf('Bus %d', k));
 end
fprintf('\n');
for i = 1:no_bus
 fprintf('  Bus %d ', i);
 for k = 1:no_bus
 fprintf('   %9.4f %+9.4fj', real(Y(i,k)), imag(Y(i,k)));
 end
 fprintf('\n');
 end
fprintf('============================================================================\n');
 Ybus = Y

 %% iteration counter
iter = 0;
del_hist = [];
delta_hist = [];

 %% Gauss-Seidel sweep
del = 1;
```

```
while del >= 0.000170
 iter = iter + 1;
 del = 0;
 for i = 2:no_bus
 sum_term = Y(i,:)*Vc.' - Y(i,i)*Vc(i);

 if isPV(i)
 Q_new = -imag( conj(Vc(i)) * (Y(i,:)*Vc.') );
 V_new = ( (P(i) - 1j*Q_new)/conj(Vc(i)) - sum_term ) / Y(i,i);
 delta_new = angle(V_new);
 del = max([del, abs(delta_new - delta(i))]);
 delta(i) = delta_new;
 Q(i) = Q_new;
 else % (PQ bus)
 V_new = ( (P(i) - 1j*Q(i))/conj(Vc(i)) - sum_term ) / Y(i,i);
 V_mag_new = abs(V_new);
 delta_new = angle(V_new);
 del = max([del, abs(V_mag_new - V(i)), abs(delta_new - delta(i))]);
 V(i) = V_mag_new;
 delta(i) = delta_new;
 end
 Vc(i) = V(i) * exp(1j*delta(i));
 if ~isfinite(V(i)) || ~isfinite(delta(i))
 error(['Divergence at bus %d, iteration %d (value became ' ...
 'NaN/Inf). Double check the R/X and P/Q/V/delta you ' ...
 'entered -- a stray inf in the wrong field is the ' ...
'usual cause.'], i, iter);
 end
 end
 % iteration's result save
 del_hist(iter) = del;
 delta_hist(iter,:) = delta*180/pi;
 end

 %% slack bus eqn
S1 = Vc(1) * conj( Y(1,:)*Vc.' );
P(1) = real(S1);
Q(1) = imag(S1);
fid = fopen('load_flow_result.txt', 'w');
fprintf(fid, 'GAUSS-SEIDEL LOAD FLOW RESULT\n');
fprintf(fid, 'Result found at iteration number: %d\n\n', iter);

fprintf(fid, 'BUS ADMITTANCE MATRIX (YBus) in pu\n');
for i = 1:no_bus
 for k = 1:no_bus
 fprintf(fid, '%9.4f %+9.4fj   ', real(Y(i,k)), imag(Y(i,k)));
 end
 fprintf(fid, '\n');
 end
fprintf(fid, '\n');

 fprintf(fid, 'Bus V(pu) Angle(deg) P(pu) Q(pu)\n');
for i = 1:no_bus
 fprintf(fid, '%3d %8.4f %10.4f %9.4f %9.4f\n', ...
 i, V(i), delta(i)*180/pi, P(i), Q(i));
 end

 fprintf(fid, '\nLINE FLOWS S(i,j) = P(i,j) + jQ(i,j) (pu)\n');
for i = 1:no_bus
 for k = i+1:no_bus
 if Y(i,k) ~= 0
 y_ik = -Y(i,k);
 Sik = Vc(i) * conj( (Vc(i)-Vc(k)) * y_ik );
 Ski = Vc(k) * conj( (Vc(k)-Vc(i)) * y_ik );
 fprintf(fid, '%d -> %d : %9.4f + j%9.4f\n', i, k, real(Sik), imag(Sik));
 fprintf(fid, '%d -> %d : %9.4f + j%9.4f\n', k, i, real(Ski), imag(Ski));
 end
 end
 end

fprintf(fid, '\nITERATION HISTORY (angles in degree)\n');
fprintf(fid, 'Iter');
for i = 2:no_bus
 fprintf(fid, '   delta%d(deg)', i);
 end
fprintf(fid, '   max change (del)\n');
for n = 1:iter
 fprintf(fid, '%3d ', n);
```

```
 for i = 2:no_bus
 fprintf(fid, '%14.4f', delta_hist(n,i));
 end
 fprintf(fid, '%18.6f\n', del_hist(n));
 end
fclose(fid);

 fprintf('\nConverged at iteration %d. Results saved to load_flow_result.txt\n', iter);

 %% Convergence graph
figure;
semilogy(1:iter, del_hist, '-o', 'LineWidth', 1.5, 'MarkerFaceColor', 'b');
hold on;
plot([1 iter], [0.000170 0.000170], 'r--', 'LineWidth', 1.2);
text(1, 0.000170, '  Tolerance = 0.00017', 'Color', 'r', 'VerticalAlignment', 'bottom');
hold off;
grid on;
xlabel('Iteration number');
ylabel('Maximum change, del');
legend('del (largest change)', 'Tolerance', 'Location', 'northeast');
title('Convergence of Gauss-Seidel Load Flow');
set(gca, 'XTick', 1:iter);

 %%  Bus voltage angles vs iterations
figure;
plot(1:iter, delta_hist(:,2:no_bus), '-o', 'LineWidth', 1.5);
grid on;
xlabel('Iteration number');
ylabel('Angle (degree)');
title('Bus voltage angles through the iterations');
legend(arrayfun(@(k) sprintf('\\delta%d', k), 2:no_bus, 'UniformOutput', false), 'Location', 'best');
set(gca, 'XTick', 1:iter);
```

Example (Table 1 and Table 2)

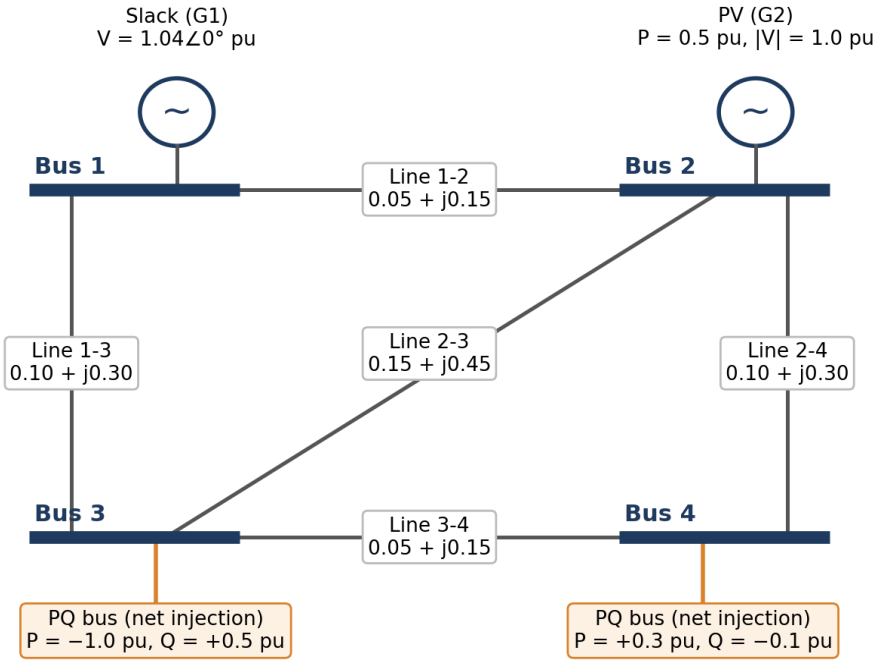


Figure 1: One-line diagram of the 4-bus system

Bus	Type	P (pu)	Q (pu)	|V| (pu)	δ
1	Slack	unknown	unknown	1.04	0
2	PV	0.5	unknown	1.00	unknown
3	PQ	−1.0	0.5	unknown	unknown
4	PQ	0.3	−0.1	unknown	unknown

Table 1: Bus data

Line	R (pu)	X (pu)	y = 1/(R + jX) (pu)
1-2	0.05	0.15	2 − j6
1-3	0.10	0.30	1 − j3
2-3	0.15	0.45	0.6667 − j2
2-4	0.10	0.30	1 − j3
3-4	0.05	0.15	2 − j6
1-4	—	—	no line (0)

Table 2: Line data.

Input

```
Total no of bus is: 4
--- Bus 1 ---
P(1) in pu (type inf if unknown): inf
Q(1) in pu (type inf if unknown): inf
V(1) in pu (type inf if unknown): 1.04
delta(1) in rad (type inf if unknown): 0
--- Bus 2 ---
P(2) in pu (type inf if unknown): 0.5
Q(2) in pu (type inf if unknown): inf
V(2) in pu (type inf if unknown): 1
delta(2) in rad (type inf if unknown): inf
--- Bus 3 ---
P(3) in pu (type inf if unknown): -1
Q(3) in pu (type inf if unknown): 0.5
V(3) in pu (type inf if unknown): inf
delta(3) in rad (type inf if unknown): inf
--- Bus 4 ---
P(4) in pu (type inf if unknown): 0.3
Q(4) in pu (type inf if unknown): -0.1
V(4) in pu (type inf if unknown): inf
delta(4) in rad (type inf if unknown): inf
--- Line 1-2 (type inf,inf if no line) ---
R(1,2) in pu: 0.05
X(1,2) in pu: 0.15
--- Line 1-3 (type inf,inf if no line) ---
R(1,3) in pu: 0.10
X(1,3) in pu: 0.30
--- Line 1-4 (type inf,inf if no line) ---
R(1,4) in pu: inf
X(1,4) in pu: inf
--- Line 2-3 (type inf,inf if no line) ---
R(2,3) in pu: 0.15
X(2,3) in pu: 0.45
--- Line 2-4 (type inf,inf if no line) ---
R(2,4) in pu: 0.10
X(2,4) in pu: 0.30
--- Line 3-4 (type inf,inf if no line) ---
R(3,4) in pu: 0.05
X(3,4) in pu: 0.15
```

Result

```
==================== BUS ADMITTANCE MATRIX (YBus) in pu ====================
                      Bus 1                Bus 2                Bus 3                Bus 4
  Bus 1       3.0000   -9.0000j     -2.0000   +6.0000j     -1.0000   +3.0000j      0.0000   +0.0000j
  Bus 2      -2.0000   +6.0000j      3.6667  -11.0000j     -0.6667   +2.0000j     -1.0000   +3.0000j
  Bus 3      -1.0000   +3.0000j     -0.6667   +2.0000j      3.6667  -11.0000j     -2.0000   +6.0000j
  Bus 4       0.0000   +0.0000j     -1.0000   +3.0000j     -2.0000   +6.0000j      3.0000   -9.0000j
============================================================================
```

Fig.1. Ybus displayed in the command window

```
GAUSS-SEIDEL LOAD FLOW RESULT
Result found at iteration number: 10


BUS ADMITTANCE MATRIX (YBus) in pu
   3.0000   -9.0000j     -2.0000   +6.0000j     -1.0000   +3.0000j      0.0000   +0.0000j
  -2.0000   +6.0000j      3.6667  -11.0000j     -0.6667   +2.0000j     -1.0000   +3.0000j
  -1.0000   +3.0000j     -0.6667   +2.0000j      3.6667  -11.0000j     -2.0000   +6.0000j
   0.0000   +0.0000j     -1.0000   +3.0000j     -2.0000   +6.0000j      3.0000   -9.0000j


Bus V(pu) Angle(deg) P(pu) Q(pu)
  1   1.0400     0.0000    0.2413    0.2558
  2   1.0000     1.6473    0.5000   -0.5278
  3   1.0290    -5.7182   -1.0000    0.5000
  4   1.0169    -1.4649    0.3000   -0.1000


LINE FLOWS S(i,j) = P(i,j) + jQ(i,j) (pu)
1 -> 2 :   -0.0953 + j   0.3120
2 -> 1 :    0.1002 + j  -0.2972
1 -> 3 :    0.3367 + j  -0.0562
3 -> 1 :   -0.3259 + j   0.0885
2 -> 3 :    0.2502 + j  -0.1289
3 -> 2 :   -0.2383 + j   0.1645
2 -> 4 :    0.1502 + j  -0.1015
4 -> 2 :   -0.1469 + j   0.1113
3 -> 4 :   -0.4351 + j   0.2468
4 -> 3 :    0.4469 + j  -0.2114


ITERATION HISTORY (angles in degree)
Iter   delta2(deg)   delta3(deg)   delta4(deg)   max change (del)
  1         2.6747       -4.8529       -0.5128          0.084700
  2         2.0860       -5.1289       -0.9230          0.010274
  3         1.9082       -5.3775       -1.1509          0.004339
  4         1.7993       -5.5197       -1.2828          0.002482
  5         1.7350       -5.6037       -1.3601          0.001466
  6         1.6968       -5.6536       -1.4059          0.000871
  7         1.6740       -5.6834       -1.4331          0.000520
  8         1.6604       -5.7012       -1.4494          0.000311
  9         1.6522       -5.7118       -1.4591          0.000186
 10         1.6473       -5.7182       -1.4649          0.000112
```

Fig.2. Text file result

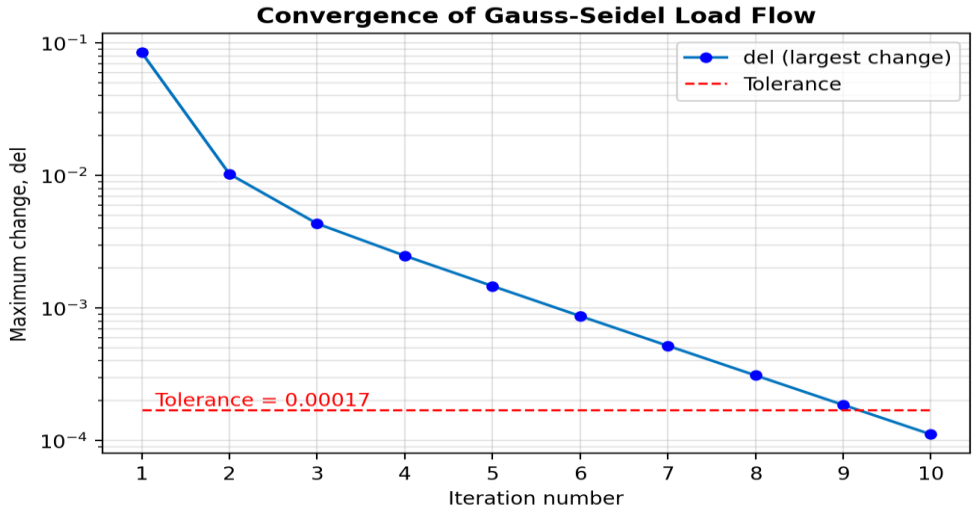


Figure 2: Convergence graph produced by the program

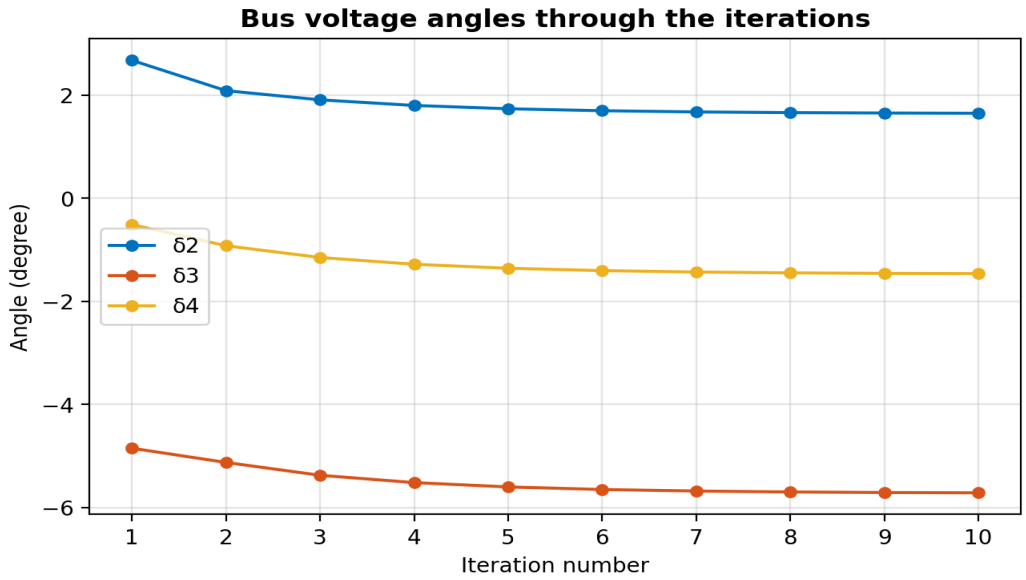


Figure 3: Bus voltage angles through the iterations

Discussion

A MATLAB program for Gauss-Seidel load flow was developed according to the procedure of the manual: bus data are stored in row vectors, unknown quantities are entered as inf, Ybus is formed with two nested loops, PV and PQ buses are identified from the data, and the results are written to a text file with fprintf. For the 4-bus system the solution converged in 10 iterations. The hand calculation of Ybus and of the whole first iteration (including del = 0.0847) agrees exactly with the program.

The slack bus supplies 0.2413 + j0.2558 pu. Bus 2 absorbs 0.5278 pu reactive power to hold its voltage at 1.0 pu. Bus 3, which takes the largest real power, has the most negative angle (−5.72°); it also receives 0.5 pu reactive power, so its voltage (1.029 pu) is the highest of the PQ buses. Real power flows from higher to lower angle: bus 2 (1.65°) sends power to buses 1, 3 and 4, and line 3-4 is the most heavily loaded line. The total loss is 0.0427 + j0.1280 pu.

The convergence graph shows del falling almost linearly on a logarithmic scale, which is typical of the Gauss-Seidel method: simple and light on memory, but slow for large systems. An acceleration factor (1.4–1.6) or the Newton-Raphson method would converge in fewer iterations. The tolerance used (0.00017) is slightly looser than the manual's 0.0001; a smaller tolerance gives a closer power balance at the cost of a few more iterations. The safety checks in the program (unknown P at a non-slack bus, isolated bus, divergence) stop the run with a clear message instead of producing meaningless results.